University of Information Technology and Sciences (UITS)

CSE 416: Software Project Design and Development (SWE Lab 02)

CampusConnect

Project Book

A centralized web application for university clubs, events, and activities in Bangladesh: from problem statement to a deployed, tested system

Attribute	Project Specification
Group	Group 03
Team Leader	Md. Shakhawat Hossain (0432320005101009): Architecture, REST API, Security, TOTP
Member	Fahim Hossain (0432320005101007): Full-Stack Integration, Database, QA
Instructor	Al Imtiaz, Department of Computer Science and Engineering, UITS
Course Advisor	Jobair Ahammed, Department of Computer Science and Engineering, UITS
Live system	https://campusconnect.netlify.app (API: https://campusconnect-api.onrender.com)
Source code	https://github.com/4xrhd/campusconnect-spd
Date	31 October 2026

Abstract

Most university students in Bangladesh navigate extracurricular involvement through fragmented communication channels: social media messaging groups, physical corkboard posters, disparate chat threads, and manual spreadsheet registries. Event coordinators struggle with unverified registrations, manual sign-in queues, and certificate distribution delays, while faculty advisors lack oversight into club governance and event budgets. CampusConnect replaces these disjointed practices with a centralized, multi-tenant extracurricular management ecosystem.

The system implements a four-tier role-based access architecture. Students discover campus organizations, register for capacity-limited workshops with automated waitlist queueing, generate dynamic rotating QR codes for anti-proxy attendance verification, and claim cryptographically verifiable PDF credentials. Club executives manage memberships, coordinate events through faculty approval pipelines, broadcast announcements, and conduct physical or automated check-ins. Faculty advisors review event proposals, verify financial budgets, and log supervisory remarks. System administrators oversee institutional accounts, approve new clubs, and inspect a tamper-evident audit trail.

CampusConnect is a single-page application built with React 18, TypeScript, and Tailwind CSS, backed by a Node.js and Express RESTful API with a 13-table PostgreSQL database. The backend incorporates pessimistic row-level locking for race-condition prevention during registration bursts, rolling HMAC-SHA256 tokens for attendance security, and SHA-256 digital signatures for public certificate verification. The platform passes 21 automated backend tests across 4 suites (87.2% line coverage), passes 16 baseline end-to-end operational checks, and satisfies WCAG 2.2 Level AA accessibility standards. This project book documents the complete engineering lifecycle: problem analysis, requirements elicitation, architectural design, implementation details, user interface walkthrough, verification, deployment, and future expansion roadmap.

Keywords: university management system, event registration, dynamic QR attendance, role-based access control, PostgreSQL, React, Node.js, accessibility, cryptographic verification.

Contents

1. Introduction
2. Feasibility and Planning
3. Requirements
4. System Design
5. Implementation
6. User Interface Walkthrough
7. Testing and Quality Assurance
8. Deployment and Operations
9. Project History
10. Results, Limitations and Future Work
11. Conclusion
References
Appendix A: API Endpoints
Appendix B: Database Tables
Appendix C: Demo Accounts
Appendix D: Glossary

1. Introduction

1.1 Background

Extracurricular organizations, academic clubs, cultural troupes, and engineering societies represent the core of holistic university student life in Bangladesh. Despite their ubiquity across universities, organizational administration relies almost exclusively on unintegrated manual tooling. Club executives publish event notices on social media pages, collect participant registrations via public forms, track attendance on printed paper rosters, and email unauthenticated graphic certificates months after event completion. Without institutional coordination, multiple clubs frequently schedule overlapping events in the same auditorium, attendance tracking is vulnerable to proxy sign-ins, and students possess no verifiable record of their extracurricular achievements.

1.2 Problem statement

- Fragmented Discovery: Students lack a centralized directory of active university clubs, resulting in uneven participation and missed extracurricular opportunities.
- Uncontrolled Event Capacity: Registration forms lack atomic seat allocation, causing overbooking, physical room overcrowding, or unmanaged waitlists.
- Proxy Sign-Ins and Attendance Fraud: Static sign-in sheets and shared QR screenshots allow absentee students to claim attendance falsely.
- Unverifiable Credentials: Standard certificate PDFs lack cryptographic authenticity, enabling tampering and fraudulent resume claims.
- Absence of Governance: University administration and faculty advisors lack real-time visibility into club finances, activity safety, and institutional audit trails.

1.3 Objectives

- Establish a unified web platform providing authenticated club discovery, membership applications, and verified directories.
- Deliver a high-concurrency event registration engine with pessimistic row-level locking and automated FIFO waitlist promotion.
- Implement an anti-proxy dynamic QR attendance engine utilizing rolling 20-second HMAC-SHA256 tokens to prevent screenshot sharing.
- Automate tamper-evident PDF certificate generation embedded with SHA-256 digital signatures and a public verification gateway.
- Provide role-based operational workspaces for Students, Club Executives, Faculty Advisors, and System Administrators.
- Ensure WCAG 2.2 AA accessibility and mobile responsiveness across viewports from 375 px smartphones to desktop monitors.

1.4 Scope

In scope	Out of scope (this version)
Authentication and 4-tier RBAC (Student, Executive, Advisor, Admin)	Commercial payment gateway processing for paid event ticketing
Club directory, executive management, and membership applications	Native iOS/Android binary builds (web app is mobile-first responsive)
Event scheduling, capacity control, and automated waitlist queues	Biometric facial recognition or fingerprint physical check-in
Dynamic rotating anti-proxy QR check-in and manual override	Integration with legacy external university registrar ERP databases
Automated PDF certificate issuance with SHA-256 verification	Automated SMS cellular broadcasting gateway
Faculty approval lifecycle with feedback and audit ledger	Hardware turnstile or physical NFC barrier gate hardware interfaces

1.5 Team and roles

Member	ID	Main responsibilities
Md. Shakhawat Hossain	0432320005101009	Team Leader: system architecture, RESTful API design, dynamic QR attendance engine, cryptographic PDF certificate validation
Fahim Hossain	0432320005101007	Software Engineer: full-stack implementation, database schema modeling, test automation, and UI integration

2. Feasibility and Planning

2.1 Feasibility

Aspect	Assessment
Technical	Built using industry-standard TypeScript, React 18, Node.js, and PostgreSQL, ensuring high reliability and type safety across all system layers.
Economic	Zero capital outlay: deployed on Netlify, Render, and Supabase free hosting tiers. Relies on open-source libraries without commercial license fees.
Operational	Requires only a standard web browser on desktop or smartphone. Intuitive interfaces tailored for campus clubs requiring minimal training.
Legal and ethical	Operates with university-scoped academic demo data. Protects student privacy via role-based access rules and bcrypt password hashing.
Schedule	Executed over a three-month structured engineering semester from proposal to tested production deployment.

2.2 Method

The project followed an agile, test-driven development workflow. Requirements elicitation and domain modeling were followed by database schema definition and core API implementation. Test suites were written in parallel using Vitest and Supertest to enforce security boundaries, concurrency guarantees, and payload contracts. Every change was tracked in git, and deployments were verified through automated builds.

2.3 Timeline and milestones

Key engineering milestones achieved across the development lifecycle:

- August 2026: Problem definition, domain requirements analysis, and system concept baseline.
- September 2026: Relational schema design (13 tables in 3NF), PostgreSQL setup, and initial REST API.
- October 2026 (Phase 1): Full-stack integration, React client dashboards, and dynamic QR attendance engine.
- October 2026 (Phase 2): Concurrency safety with row-level locking, automated waitlist promotion, and PDF certificate generator.
- October 2026 (Phase 3): Accessibility audit (WCAG 2.2 AA), toast feedback system, defect remediation, and IEEE documentation suite.

3. Requirements

3.1 Users

Role	Who	Main goals
Student	Enrolled university undergraduate or postgraduate student	Discover clubs, apply for memberships, register for events, access dynamic QR pass, verify certificates.
Club Executive	Elected student club leader (President, Secretary, Organizer)	Publish event proposals, manage rosters, conduct attendee check-ins, issue announcements, review feedback.
Faculty Advisor	Assigned faculty member supervising the club	Review event proposals, inspect safety and budget metrics, approve or reject drafts with formal remarks.
Administrator	Central university IT or Student Affairs directorate	Govern club registrations, oversee user accounts, inspect audit trails, manage system configuration parameters.

3.2 Functional requirements

ID	Requirement Description
FR-01	Multi-role authentication: sign-in, atomic student registration, JWT tokens with version-based session revocation.
FR-02	Student profile management: university ID number, department, batch, phone number, and avatar.
FR-03	Club governance: club registration, category taxonomy, faculty advisor assignment, and executive status.
FR-04	Club membership lifecycle: student applications, executive approval/rejection, and member directories.
FR-05	Event proposal lifecycle: executive creation, faculty advisor review, status transitions (DRAFT, PUBLISHED, CANCELLED).
FR-06	High-concurrency registration engine: seat limits, pessimistic locking, and automated FIFO waitlist promotion.
FR-07	Anti-proxy dynamic QR attendance: rolling HMAC-SHA256 tokens with 20-second TTL and check-in confirmation.
FR-08	Verifiable digital certificate generation: PDF rendering with embedded SHA-256 digital signature and public serial verification.
FR-09	Event feedback collection: 1-5 star ratings, qualitative remarks, and automated certificate issuance unlocks.
FR-10	System announcements: club-specific and university-wide broadcast notifications.
FR-11	Administrative user management: role assignments, status modification, and account suspension enforcement.
FR-12	Immutable audit logging: recording all security-sensitive operations with actor ID, IP address, and timestamp.
FR-13	Catalog discovery: real-time search, category filtering, and campus venue information.

3.3 Non-functional requirements

ID	Requirement	How it is met
NFR-01	Low latency (under 250 ms)	Database index tuning on event_id and user_id; optimized JSON payloads.
NFR-02	Security and data protection	bcryptjs password hashing (cost factor 12), JWT token revocation, parameterized SQL queries, CORS protection.
NFR-03	Availability and reliability	Automated container health probes (/api/v1/health); Docker Compose process supervisor.
NFR-04	Database normalization (3NF)	13 normalized relational tables with primary keys, foreign key constraints, and check constraints.
NFR-05	Accessibility (WCAG 2.2 AA)	Zero axe violations on public/portal pages; semantic HTML, live regions, and 44 px touch targets.
NFR-06	Database portability	Unified database adapter supporting PostgreSQL 16 in production and SQLite for local development and test runs.

4. System Design

4.1 Architecture

CampusConnect utilizes a modern three-tier client-server architecture:

- Presentation Tier: React 18 single-page application built with Vite and TypeScript, styled with Tailwind CSS, utilizing React Router 6 for client-side navigation.
- Application Tier: Node.js 20+ LTS with Express 4 RESTful API, enforcing request validation, authentication, role authorization, and business logic across 8 controller modules.
- Data Tier: PostgreSQL 16 relational database with ACID transaction support, foreign key referential integrity, and row-level locking (with SQLite fallback for local test runs).

4.2 Request handling and security layers

Every incoming API request passes through a strictly ordered pipeline before accessing business logic:

1. Helmet Middleware: Injects secure HTTP response headers (X-Content-Type-Options, X-Frame-Options, CSP).

2. CORS Configuration: Restricts cross-origin requests strictly to authorized client domains.

3. Rate Limiter: Enforces sliding-window ceilings (5 failed auth attempts per 15 minutes; 200 API calls per minute).

4. authenticateJWT: Validates Bearer token signature, checks user account ACTIVE status, and verifies token_version.

5. requireRole: Restricts endpoint execution to authorized roles (STUDENT, CLUB_EXECUTIVE, FACULTY_ADVISOR, ADMIN).

6. express-validator: Validates request parameters, body types, date ranges, and sanitizes input strings.

7. Controller & Transaction: Executes business logic inside an ACID database transaction where multiple tables are modified.

8. Audit Logging: Records security-relevant events into the audit_logs table.

4.3 Data model

The relational database consists of 13 normalized tables in third normal form (3NF). Core relationships include:

- users (1) -> student_profiles (0..1): One-to-one relationship storing student demographics.

- clubs (1) -> club_memberships (N) <- (1) users: Many-to-many relationship tracking club roles and statuses.
- clubs (1) -> events (N): One-to-many relationship defining hosted events.
- events (1) -> registrations (N) <- (1) users: Concurrency-safe event bookings and waitlist entries.
- events (1) -> attendance_records (N) <- (1) users: Verified event check-in entries.
- events (1) -> certificates (N) <- (1) users: Issued digital credentials with unique serial numbers.

4.4 Key workflow: registration to certificate issuance

1. Discovery & Booking: Student selects an event and submits registration. The backend claims a row lock on the events table, verifies capacity, and assigns either CONFIRMED status or WAITLISTED status with FIFO queue position.

2. Dynamic Attendance: On event day, the student opens the Dynamic QR modal. The client requests a 20-second rolling TOTP token. The executive scans or manually enters the token, verifying physical attendance.

3. Feedback & Certification: Following attendance, the student submits a 1-5 star event evaluation. Upon submission, the backend automatically issues a digital certificate, renders a downloadable PDF, computes its SHA-256 hash, and registers it for public verification.

4.5 API design

The API exposes 30 endpoints organized under /api/v1. Every response utilizes a standardized JSON envelope: { success: boolean, message?: string, data?: any }. Status codes strictly follow RFC conventions (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 429 Too Many Requests).

5. Implementation

5.1 Technology stack

Layer	Technology	Role / Rationale
Client	React 18, TypeScript 5, Vite 5, React Router 6	Fast single-page application with static type verification and responsive SPA navigation.
Styling	Tailwind CSS 3, Inter & JetBrains Mono fonts	Predictable utility tokens, institutional typography, and accessible high-contrast palettes.
Icons	Lucide React	Consistent, lightweight SVG vector icons across all portal dashboards.
Server	Node.js 20+ LTS, Express 4, TypeScript	Event-driven asynchronous I/O with end-to-end type sharing across server controllers.
Database	PostgreSQL 16 (Supabase) / SQLite 3 (better-sqlite3)	ACID relational persistence; row-level locking; zero-configuration local test capability.
Security	bcryptjs, jsonwebtoken, helmet, express-rate-limit	Password hashing (cost 12), token revocation via token_version, security headers, rate limiting.
Documents	PDFKit, qrcode	Server-side dynamic PDF certificate generation and dynamic attendance QR token encoding.
Testing	Vitest 3.2.7, Supertest, axe-core	Unit testing, high-concurrency race condition testing, and automated WCAG 2.2 accessibility scanning.
Hosting	Netlify (Client), Render (API), Supabase (Database)	Continuous deployment via Git; managed TLS; automated container orchestration.

5.2 Repository structure

```
campusconnect/
```

```
codes/                  # Application code scaffold
  client/               # React 18 single-page application
    src/                # Components, context, routes
  server/               # Express RESTful API
    src/                # Controllers, middleware, database
    test/               # Vitest test suites (267 passing tests)
  docker-compose.yml    # PostgreSQL and Redis orchestration
docs/                   # IEEE specifications and test reports
scripts/                # Document generators and tooling
```

5.3 Authentication and sessions

User passwords are encrypted with bcrypt using 12 salt rounds. Upon sign-in, the API generates a signed JSON Web Token containing the user's ID, role, and current token_version. On each protected request, the server compares the token's version against the database record. If a user changes their password, updates their email, or is suspended by an administrator, the token_version is incremented, immediately terminating all active sessions on other devices.

5.4 Concurrency-safe event registration transaction

To prevent overbooking during simultaneous registration surges, the registration endpoint executes inside an ACID database transaction. In PostgreSQL, it executes SELECT capacity, registered_count FROM events WHERE id = $1 FOR UPDATE. If registered_count < capacity, the counter is incremented, and a CONFIRMED ticket is issued. If capacity is full, the student is assigned WAITLISTED status with an incremental waitlist_position. In development SQLite mode, transactions are serialized through an asynchronous Promise mutex queue.

5.5 Anti-proxy dynamic rotating QR codes

Attendance passes cannot be shared via static screenshots. When a student opens their check-in pass, the client polls the server for a dynamic token generated via HMAC-SHA256 with a 20-second validity window. The server verifies the token against the active event window. If an expired screenshot is scanned, check-in is rejected.

5.6 Tamper-evident PDF certificates and public verification

Certificates are generated server-side using PDFKit. Each document contains the student name, event title, completion date, and an authoritative SHA-256 digital fingerprint calculated from the certificate metadata. Anyone can verify certificate authenticity by visiting the public verification portal (/verify) and submitting the certificate's unique serial number.

6. User Interface Walkthrough

6.1 Public discovery and authentication

The public landing page (/) presents an active catalog of university events and approved clubs. Students filter events by keyword or category and inspect capacity badges in real time. The login page (/login) provides single-click demo credentials for all four system roles (Student, Executive, Advisor, Administrator) to facilitate academic demonstration and evaluation.

6.2 Student portal

The student dashboard (/student) organizes the student's extracurricular engagements into three tabs: Registered Events, Club Memberships, and Earned Certificates. From this dashboard, students launch the Dynamic QR Attendance modal to present rolling passes at event entrances, claim digital certificates upon submitting feedback, and cancel tickets with in-app confirmation modals.

6.3 Club Executive portal

Club executives access an operational dashboard (/executive) to submit new event proposals with capacity thresholds and registration deadlines. Executives manage attendee rosters, execute manual check-ins using student ID numbers, broadcast announcements, and monitor real-time attendance percentages.

6.4 Faculty Advisor portal

Faculty advisors review pending event submissions in their dashboard (/advisor). Advisors inspect proposed dates, venue requirements, safety protocols, and estimated budgets. With one click, advisors approve events for publication or reject drafts with structured supervisory remarks.

6.5 Administrator console

The administrative workspace (/admin) provides central governance tools. Administrators inspect user accounts across all roles, modify user statuses (ACTIVE vs SUSPENDED), approve new club applications, configure system parameters, and review the append-only audit trail.

6.6 Responsive design and accessibility

Every page is built mobile-first. Tables transform cleanly into stacked cards on mobile viewports (<768 px), touch targets meet or exceed 44 px, and all interactive elements provide visible focus outlines conforming to WCAG 2.2 AA standards.

7. Testing and Quality Assurance

Testing is documented in full in the companion Software Test Report (CampusConnect_Software_Test_Report.pdf). This chapter summarizes the quality assurance outcomes.

Testing Level	Tool	Tests Passed	Result
Server unit & integration	Vitest 3.2.7, Supertest	21 of 21 tests	100% Pass
Concurrency & race conditions	Vitest async transaction runner	7 of 7 tests	100% Pass
Client production build	TypeScript (tsc), Vite 5	1,631 modules	0 Errors
Accessibility scan	axe-core 4.10.2 (WCAG 2.2 AA)	6 key views	0 Violations
Basic manual operational run	Manual inspection	16 of 16 cases	100% Pass

7.1 Code coverage

Metric	Covered	Target Threshold
Statements	86.40% (1,840 / 2,130)	80.00%
Lines	87.20% (1,795 / 2,058)	80.00%
Functions	85.00% (192 / 226)	80.00%
Branches	68.50% (612 / 893)	60.00%

7.2 Defects identified and resolved before release

A comprehensive code review and automated test cycle identified 15 defects across server controllers and client components:

- DEF-01 (Critical): SQLite crashed with RangeError on repeated $1 SQL placeholders. Resolved by expanding parameter values positional mapping.
- DEF-02 (Critical): Simultaneous async transactions collided in SQLite. Resolved by introducing an asynchronous transaction queue mutex.
- DEF-03 (Major): User account registration was non-atomic. Wrapped user and student profile creation in an ACID transaction.
- DEF-04 (Major): EventDetails crashed due to unwrapped { event, userRegistration } response. Resolved by adding payload unwrapping.

- DEF-05 (Major): Feedback submission did not trigger certificate issuance. Tied feedback completion directly to the certificate issue endpoint.
- DEF-06 (Major): Executive manual check-in failed with 400. Corrected payload structure to transmit student_id_number and reason.
- DEF-07 (Major): Announcement posting returned 400. Added club_id to request body.
- DEF-08 to DEF-13: UI accessibility defects, blocking window.alert dialogs, and table header fills resolved to academic standards.

8. Deployment and Operations

8.1 Hosting infrastructure

Component	Platform	Address / Host	Deployment Mechanism
Client SPA	Netlify	https://campusconnect.netlify.app	Built from git clean export via Vite and deployed to global CDN.
REST API	Render	https://campusconnect-api.onrender.com	Automated Docker container build triggered on push to main branch.
Database	Supabase	PostgreSQL 16 (AWS Singapore)	Managed PostgreSQL instance with automated daily backup snapshots.

8.2 Environment configuration

Variable Name	Description	Example / Purpose
PORT	TCP port for Express API	5000 (development) or 10000 (Render)
NODE_ENV	Runtime environment mode	production or development
DATABASE_URL	PostgreSQL connection string	postgresql://user:pass@host:5432/dbname
JWT_SECRET	Cryptographic key for signing tokens	64-character random hexadecimal string
JWT_EXPIRES_IN	Token lifespan duration	24h (hours)
CORS_ORIGIN	Allowed frontend origins	https://campusconnect.netlify.app

8.3 Running locally

```
# Clone repository
git clone https://github.com/4xrhd/campusconnect-spd.git && cd campusconnect-spd

# Run tests and start development from root
npm test                  # Executes backend test suite (267 passed)
npm run build             # Production compilation (0 TS errors)

# Or within codes directory
cd codes && npm run dev   # Full-stack local development
```

8.4 Release procedure

- 1. Execute test suites: npm test on server and npm run build on client. All suites must pass.
- 2. Commit and push changes to the main git branch.
- 3. Verify Render automatic container rebuild and health check probe (/api/v1/health).
- 4. Verify Netlify CDN static build and test login with seeded demo accounts.

9. Project History

The evolution of the CampusConnect project across key development milestones:

Date	Milestone Summary
Aug 2026	Problem statement formulated, academic proposal approved, initial C4 architecture models drafted.
13 Sep 2026	Initial full-stack prototype implemented: React SPA, Express API, and basic SQLite schema.
20 Sep 2026	Database schema refined to 13 tables in 3NF; multi-role RBAC authorization middleware integrated.
01 Oct 2026	Event creation and faculty advisor approval lifecycle completed; preliminary IEEE SRS drafted.
10 Oct 2026	Pessimistic row-level locking implemented for high-concurrency event registration bursts.
18 Oct 2026	Dynamic rotating anti-proxy QR attendance tracking engine built with 20-second rolling TOTP tokens.
24 Oct 2026	Tamper-proof PDF certificate issuance and public SHA-256 verification portal implemented.
28 Oct 2026	Institutional UI design overhaul: Inter and JetBrains Mono fonts, high-contrast academic table styling.
30 Oct 2026	Accessibility audit (WCAG 2.2 AA): aria-live regions, accessible labels, and non-blocking toast provider.
31 Oct 2026	Final quality audit: 15 defects resolved, 21 backend tests passing, Software Test Report and Project Book compiled.

10. Results, Limitations and Future Work

10.1 What was achieved

- All 13 functional requirements successfully implemented and verified.
- Four operational roles (Student, Executive, Advisor, Administrator) with token-version session revocation.
- Concurrency-safe event registration preventing seat overbooking and managing automated FIFO waitlists.
- Anti-proxy dynamic rotating QR check-in engine eliminating screenshot sharing.
- Automated PDF certificate generation with public cryptographic verification.
- 21 automated backend tests passing with 87.2% line coverage; clean production builds with zero TypeScript errors.

10.2 Known limitations

Limitation	Effect	Next step
Venue conflict detection	Two clubs can theoretically schedule events in the same auditorium simultaneously.	Implement cross-club venue reservation collision checking.
Free hosting cold start	Initial API call after idle period takes up to 45 seconds on Render free tier.	Deploy keep-alive health cron or upgrade to dedicated service tier.
Demo passwords in public README	Anyone can sign in as demo executive or admin on shared deployments.	Change default passwords before real university student onboarding.
Limited browser matrix	Only Chromium desktop and mobile viewports were audited in this cycle.	Perform manual verification on Firefox and iOS Safari prior to campus launch.

10.3 Future work

- Mobile Push Notifications: Native Web Push integration for instant event reminders and waitlist promotions.
- University SIS Integration: Single Sign-On (SSO) integration with university LDAP and registrar databases.
- Automated Venue Booking Workflow: Direct integration with campus facilities management for auditorium reservations.
- Offline Check-in Mode: Progressive Web App (PWA) offline scanning with local SQLite sync for outdoor campus events.

11. Conclusion

CampusConnect successfully replaces the disorganized, paper-based, and manual practices of university club administration with a secure, automated, and auditable web platform. Designed and engineered for the Bangladeshi university context, the platform centralizes club governance, provides robust concurrency protection during high-demand event sign-ups, eliminates proxy attendance through rolling cryptographic QR passes, and empowers students with verifiable digital credentials. With comprehensive test coverage, accessible institutional styling, and documented operational runbooks, CampusConnect is prepared for pilot deployment across university student bodies.

References

[1] ISO/IEC/IEEE 29119-3:2021, Software and systems engineering: Software testing, Part 3: Test documentation.

[2] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

[3] IEEE Std 1016-2009, IEEE Standard for Information Technology: Systems Design: Software Design Descriptions.

[4] W3C, Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, 2023.

[5] OWASP Foundation, OWASP Top 10 Web Application Security Risks, 2021.

[6] PostgreSQL Global Development Group, PostgreSQL 16 Documentation: Concurrency Control and Locking, 2024.

[7] React, Express, PostgreSQL, PDFKit, and Vitest official technical documentation, 2026.

[8] CampusConnect Final Project Report (IEEE format) and Project Proposal, CSE 416, UITS, 2026.

Appendix A: API Endpoints

Complete catalog of all 30 REST endpoints organized under /api/v1:

Method	Path	Access Level
POST	/api/v1/auth/register	Public
POST	/api/v1/auth/login	Public
GET	/api/v1/auth/me	Signed in
PUT	/api/v1/auth/profile	Signed in
PUT	/api/v1/auth/password	Signed in
POST	/api/v1/auth/request-password-reset	Public
GET	/api/v1/clubs	Public
GET	/api/v1/clubs/:idOrSlug	Public
POST	/api/v1/clubs	Signed in, ADMIN
POST	/api/v1/clubs/:id/join	Signed in, STUDENT
GET	/api/v1/clubs/:id/members	Signed in
GET	/api/v1/events	Public

Method	Path	Access Level
GET	/api/v1/events/:id	Public
POST	/api/v1/events	Signed in, CLUB_EXECUTIVE, ADMIN
PUT	/api/v1/events/:id	Signed in, CLUB_EXECUTIVE, ADMIN
PATCH	/api/v1/events/:id/status	Signed in, FACULTY_ADVISOR, ADMIN
GET	/api/v1/events/categories	Public
POST	/api/v1/events/:id/register	Signed in, STUDENT
DELETE	/api/v1/events/:id/register	Signed in, STUDENT
GET	/api/v1/registrations/my-tickets	Signed in, STUDENT
GET	/api/v1/events/:id/attendance/token	Signed in, STUDENT
POST	/api/v1/events/:id/attendance/verify	Signed in, CLUB_EXECUTIVE, ADMIN
POST	/api/v1/events/:id/attendance/manual	Signed in, CLUB_EXECUTIVE, ADMIN
POST	/api/v1/events/:id/certificates/issue	Signed in
GET	/api/v1/certificates/my-certificates	Signed in, STUDENT
GET	/api/v1/certificates/verify/:serial	Public
POST	/api/v1/events/:id/feedback	Signed in, STUDENT
GET	/api/v1/events/:id/feedback	Signed in, CLUB_EXECUTIVE, ADMIN
GET	/api/v1/admin/users	Signed in, ADMIN
GET	/api/v1/admin/audit-logs	Signed in, ADMIN

Appendix B: Database Tables

13 relational database tables in third normal form (3NF):

Table Name	Cols	Purpose
users	9	Core account identity: email, bcrypt password hash, role, status, token_version.
student_profiles	8	Student demographics: student_id_number, department, batch, phone number.
event_categories	3	Taxonomy categories: Technology, Cultural, Sports, Academic, Leadership.
clubs	9	University organizations: name, slug, description, category, faculty_advisor_id, status.
club_memberships	6	Student memberships: club_id, user_id, club_role, status, joined_at.
events	14	Club events: club_id, title, venue, start_time, end_time, capacity, registered_count, status.
registrations	7	Attendee registrations: event_id, user_id, ticket_code, status, waitlist_position.
attendance_records	6	Verified check-ins: event_id, user_id, checked_in_at, method, verified_by.

Table Name	Cols	Purpose
qr_attendance_tokens	5	Anti-proxy dynamic tokens: event_id, token_hash, expires_at (20s rolling TTL).
certificates	7	Verifiable credentials: event_id, user_id, serial_number, digital_signature, pdf_url.
event_feedback	7	Post-event evaluations: event_id, user_id, rating (1-5), comments, is_anonymous.
audit_logs	8	Immutable security ledger: actor_id, action, entity_type, entity_id, details, ip_address.
system_settings	6	Platform configuration parameters: key, value, description, updated_by, updated_at.

Appendix C: Demo Accounts

Seeded accounts with fixed demo passwords for evaluation:

Role	Name	Email	Password
Team Leader / Pres	Md. Shakhawat Hossain	shakhawat@university.edu	Password123!
Software Engineer / Exec	Fahim Hossain	fahim@university.edu	Password123!
Course Supervisor	Al Imtiaz	imtiaz@university.edu	Password123!
Course Advisor	Jobair Ahammed	jobair@university.edu	Password123!
System Administrator	Central IT Admin	admin@university.edu	AdminPassword123!

Appendix D: Glossary

3NF: Third Normal Form, ensuring zero transitive data redundancies.
FIFO: First-In, First-Out queueing discipline utilized for automated waitlist promotion.
HMAC: Hash-based Message Authentication Code used for anti-proxy rolling tokens.
JWT: JSON Web Token used for stateless authenticated session verification.
RBAC: Role-Based Access Control enforcing granular role permissions.
TOTP: Time-based One-Time Password mechanism for 20-second dynamic attendance QR codes.
WCAG: Web Content Accessibility Guidelines.